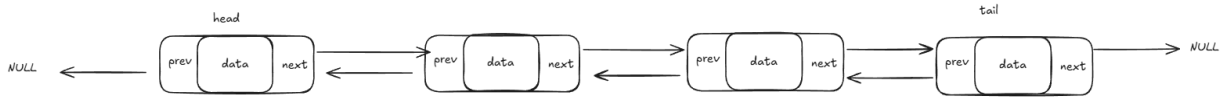
Double Linked List

No 6

Struktur Data

Double Linked List

Sebelumnya kita sudah belajar terkait apa itu Linked List serta mempraktekkan Single Linked List. Kali ini kita akan membahas Double Linked List, berikut adalah ilustrasinya:



Double Linked List adalah jenis Linked List yang dihubungkan dengan dua pointer dalam satu elemen, yaitu prev (kiri) dan next (kanan) sehingga pembacaan data dapat melintas baik maju ke depan ataupun berbalik kebelakang dengan menggunakan node next dan prev.

Kelebihan Double Linked List

Double Linked List memiliki beberapa kelebihan yang menjadikannya unggul dalam situasi tertentu. Contohnya adalah sebagai berikut:

1. Operasi menambah node baru di akhir Linked List atau menghapus node dari akhir dapat dilakukan tanpa perlu melakukan traversal seperti pada Single Linked List.
2. Struktur ini bisa digunakan dalam sistem navigasi yang memerlukan pergerakan maju dan mundur.
3. Bisa digunakan untuk mengimplementasikan Tree dalam struktur data.
4. Memungkinkan traversal dalam dua arah, baik ke depan maupun ke belakang.
5. Memudahkan navigasi, pembacaan, dan penghapusan beberapa node.
6. Mendukung akses mundur atau reverse access. Penyisipan node dapat dilakukan dengan sangat efisien.
7. Traversal dimulai dari head jika node lebih dekat ke head, dan dari tail jika node lebih dekat ke tail.

Kekurangan Double Linked List

Di sisi lain, Double Linked List juga memiliki beberapa kekurangan. Beberapa kekurangan tersebut dapat dijabarkan pada beberapa poin berikut:

1. Setiap node dalam Double Linked List memiliki pointer tambahan, sehingga membutuhkan lebih banyak memori.
2. Operasi pada Double Linked List memerlukan lebih banyak penggunaan pointer, yang menyebabkan waktu eksekusi menjadi lebih lama.
3. Pengaksesan elemen secara acak atau random tidak dapat dilakukan secara langsung.

Inisialisasi Node Double Linked List

Adapun cara untuk menginisialisasi node pada double linked list, dimana terdapat node prev yang tidak kita temukan pada single linked list. Berikut kode program menggunakan

bahasa C++:

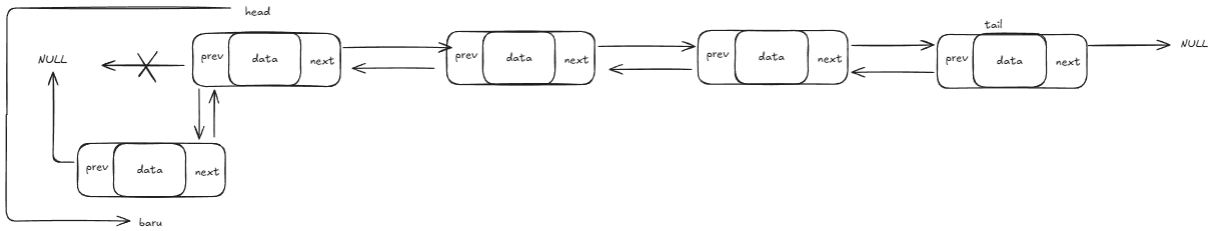
```
#include <iostream>
using namespace std;

struct mahasiswa{
   string nama;
   int nim;
   double ipk;
};
struct Node{
   mahasiswa data;
   Node *next;
   Node *prev;
};

int panjangList = 0;
```

Add First

Menambahkan node pada bagian depan atau kita sebut head pada double linked list dapat dilakukan dengan cara yang ditunjukan pada ilustrasi berikut:



Pada ilustrasi di atas, Kita bisa melihat langkah-langkah untuk menambahkan node baru di awal Double Linked List. Proses ini melibatkan pembuatan node baru, mengatur pointer next dari node baru menunjuk ke head yang lama, dan mengatur pointer prev dari head yang lama menunjuk ke node baru. Terakhir, head akan diperbarui untuk menunjuk ke node baru ini, menjadikannya elemen pertama dalam list. Berikut adalah implementasi kode program menggunakan bahasa C++:

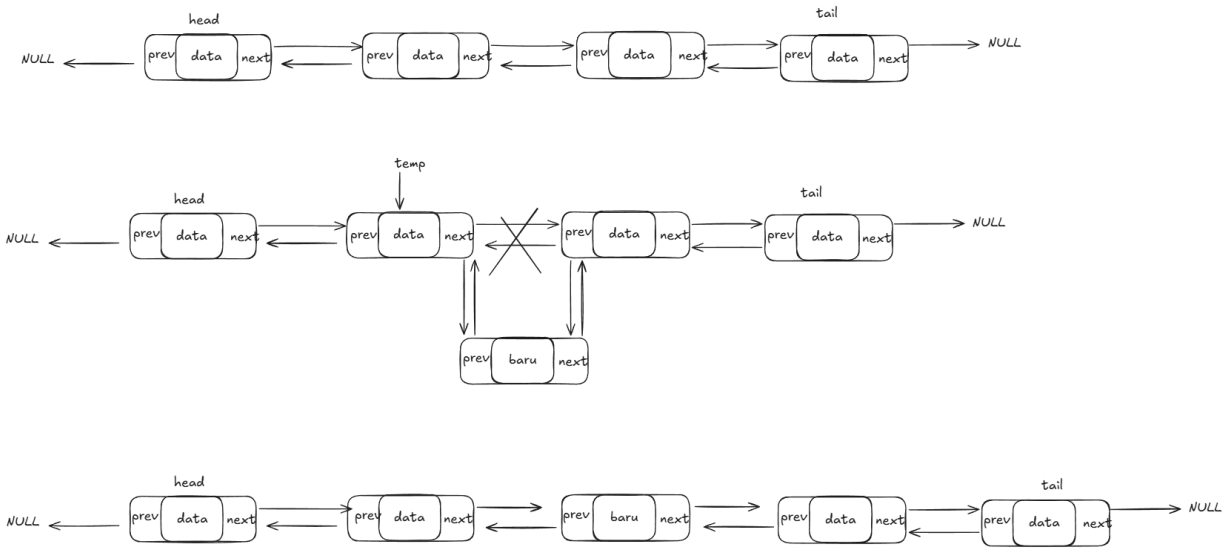
```
void insertFirst(Node *&head, Node *&tail) {
   cout << "\n[ Tambah data di awal ]" << endl;
   Node *newNode = new Node();
   cout << "Nama: "; cin.ignore(); getline(cin, newNode->data.nama);
   cout << "NIM : "; cin >> newNode->data.nim;
   cout << "IPK : "; cin >> newNode->data.ipk;
   newNode->prev = NULL;
   newNode->next = head;

   if (head == NULL && tail == NULL) {
       head = newNode;
       tail = newNode;
   } else {
       head->prev = newNode;
       head = newNode;
   }

   panjangList++;
   cout << "Data telah tersimpan." << endl;
   system("pause");
}
```

Add Specific

Menambahkan node pada bagian tertentu yakni pada bagian tengah untuk Double Linked List dapat dilakukan dengan cara yang ditunjukan pada ilustrasi berikut:



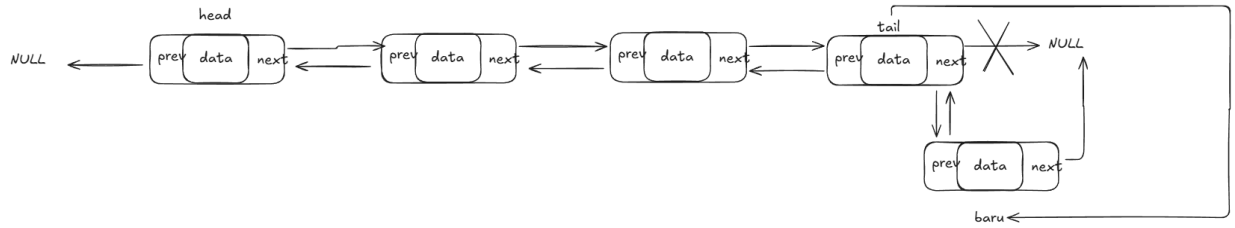
Disini kita bisa menggunakan node bantu, misal temp, node bantu ini berfungsi mencari lokasi yang ingin kita tambah, dimana dia akan berhenti di 1 node sebelum node tujuan tambah. Berikut adalah contoh implementasi menggunakan bahasa C++:

```
void insertSpecific(Node *&head, Node *&tail, int posisi) {
   if (posisi < 1 || posisi > panjangList + 1) {
       cout << "Posisi tidak valid!" << endl;
       system("pause");
       return;
   }
   if (posisi == 1) {
       insertFirst(head, tail);
   } else if (posisi == panjangList + 1) {
       insertLast(head, tail);
   } else {
       cout << "\n[ Tambah data di posisi " << posisi << " ]" << endl;
       Node *newNode = new Node();
       cout << "Nama: "; cin.ignore(); getline(cin, newNode->data.nama);
       cout << "NIM : "; cin >> newNode->data.nim;
       cout << "IPK : "; cin >> newNode->data.ipk;
       Node *temp = head;
```

```
        for (int i = 1; i < posisi - 1; i++) {
            temp = temp->next;
        }
        newNode->next = temp->next;
        newNode->prev = temp;
        temp->next->prev = newNode;
        temp->next = newNode;
        panjangList++;
        cout << "Data telah tersimpan." << endl;
        system("pause");
    }
}
```

Add Last

Menambahkan node pada bagian terakhir atau kita sebut tail pada Double Linked List dapat dilakukan dengan cara yang ditunjukan pada ilustrasi berikut:



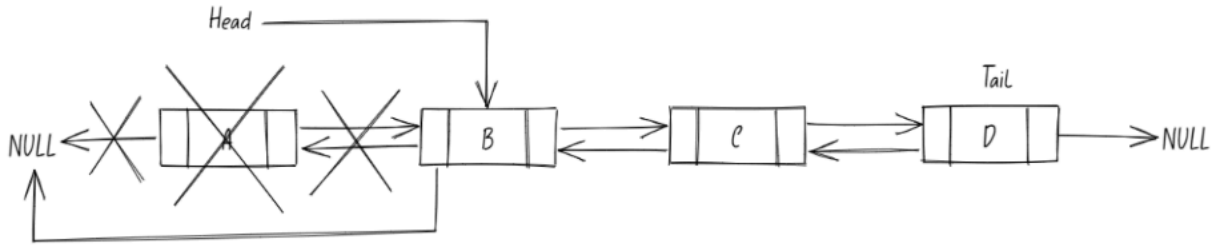
Pada ilustrasi di atas kita bisa lihat bagaimana cara menambahkan data baru di bagian akhir Double Linked List. Langkah-langkahnya biasanya meliputi membuat data baru, menghubungkan data baru ini ke data terakhir yang sudah ada (Tail), dan memperbarui penunjuk 'next' dari data terakhir serta penunjuk 'prev' dari data baru. Dengan begitu, data baru kita sekarang jadi elemen paling ujung di list. Berikut adalah implementasi kode program menggunakan bahasa C++:

```
void insertLast(Node *&head, Node *&tail){
   cout << "\n[ Tambah data di akhir ]" << endl;
   Node *newNode = new Node();
   cout << "Nama: "; cin.ignore(); getline(cin, newNode->data.nama);
   cout << "NIM : "; cin >> newNode->data.nim;
   cout << "IPK : "; cin >> newNode->data.ipk;
   newNode->prev = tail;
   newNode->next = NULL;
   if (head == NULL && tail == NULL){
       head = newNode;
       tail = newNode;
   } else {
       tail->next = newNode;
       tail = newNode;
   }
   panjangList++;

   cout << "Data telah tersimpan." << endl;
   system("pause");
}
```

Delete First

Menghapus node pada bagian depan atau kita sebut head pada Double Linked List dapat dilakukan dengan cara yang ditunjukan pada ilustrasi berikut:

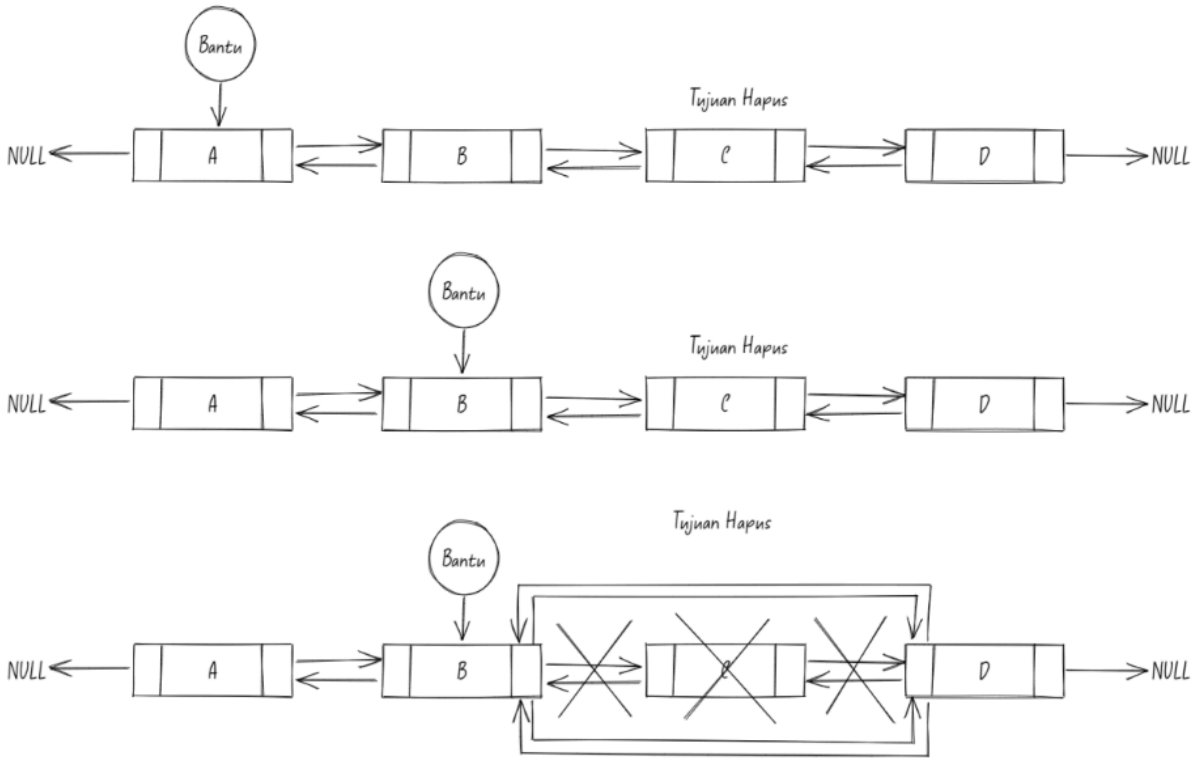


Dari ilustrasi yang ada, kita bisa melihat bagaimana proses menghapus elemen pertama atau 'head' dari Double Linked List. Intinya, kita akan menggeser 'head' ke elemen berikutnya. Setelah itu, pastikan elemen baru yang menjadi 'head' ini tidak lagi punya hubungan 'prev' ke mana pun (karena dia sudah jadi yang paling depan). Terakhir, elemen lama yang tadinya 'head' akan kita hapus dari memori. Berikut adalah implementasi kode program menggunakan bahasa C++:

```
void deleteFirst(Node *&head, Node *&tail) {
   if (head == NULL && tail == NULL) {
       cout << "LinkedList Kosong" << endl;
       system("pause");
   } else if (head->next == NULL) {
       delete head;
       head = NULL;
       tail = NULL;
       panjangList--;
       cout << "Data telah terhapus." << endl;
       system("pause");
   } else {
       Node *del = head;
       head = head->next;
       head->prev = NULL;
       delete del;
       panjangList--;
       cout << "Data telah terhapus." << endl;
       system("pause");
   }
}
```

Delete Specific

Menghapus node pada bagian tertentu yakni pada bagian tengah untuk Double Linked List dapat dilakukan dengan cara yang ditunjukan pada ilustrasi berikut:



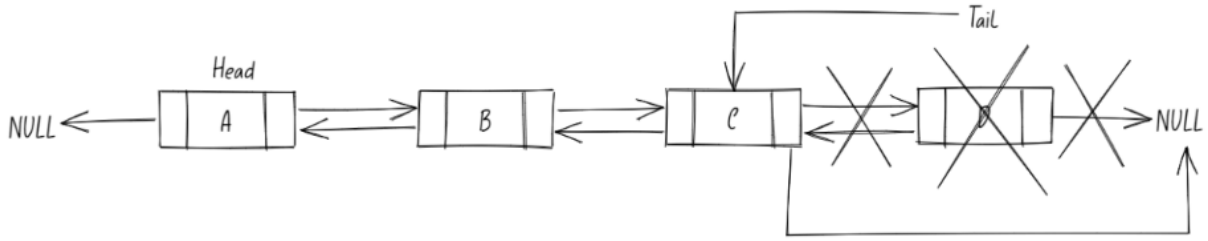
Disini kita bisa menggunakan node bantu, misal temp, node bantu ini berfungsi mencari lokasi yang ingin kita hapus, dimana dia akan berhenti di 1 node sebelum node tujuan hapus. Berikut adalah contoh implementasi menggunakan bahasa C++:

```
void deleteSpecific(Node *&head, Node *&tail, int posisi) {
   if (posisi < 1 || posisi > panjangList) {
       cout << "Posisi tidak valid!" << endl;
       system("pause");
       return;
   }
   if (posisi == 1) {
       deleteFirst(head, tail);
   } else if (posisi == panjangList) {
       deleteLast(head, tail);
   }
```

```
    else {
        Node *temp = head;
    for (int i = 1; i < posisi; i++) {
        temp = temp->next;
    }
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    delete temp;
    panjangList--;
    cout << "Data telah terhapus." << endl;
    system("pause");
    }
}
```

Delete Last

Menambahkan node pada bagian terakhir atau kita sebut tail pada Double Linked List dapat dilakukan dengan cara yang ditunjukan pada ilustrasi berikut:



Dari ilustrasi di atas, kita bisa memahami bagaimana proses menghapus elemen terakhir atau 'tail' dari Double Linked List. Caranya adalah dengan menggeser 'tail' ke elemen yang ada di depannya. Setelah itu, pastikan elemen baru yang menjadi 'tail' ini tidak lagi punya hubungan 'next' ke mana pun (karena dia sudah jadi yang paling belakang). Terakhir, elemen lama yang tadinya 'tail' akan kita hilangkan dari memori. Berikut adalah implementasi kode program menggunakan bahasa C++:

```
void deleteLast(Node *&head, Node *&tail) {
    if (head == NULL && tail == NULL){
        cout << "LinkedList Kosong" << endl;
        system("pause");
    }
```

```
    else if (head->next == NULL){
        delete head;
        head = NULL;
        tail = NULL;
        panjangList--;
        cout << "Data telah terhapus." << endl;
        system("pause");
    } else {
        Node *del = tail;
        tail = tail->prev;
        tail->next = NULL;
        delete del;
        panjangList--;

        cout << "Data telah terhapus." << endl;
        system("pause");
    }
}
```

Show/Menampilkan Data

Seperti Single Linked List, pada Double Linked List kita juga dapat menampilkan seluruh datanya. Berikut ini adalah contoh implementasi menggunakan bahasa C++:

```
void printList(Node *head) {
    if (head == nullptr) {
        cout << ">> LinkedList masih kosong <<" << endl;
        return;
    }
    cout << "\nIsi LinkedList:" << endl;
    int i = 1;
    while (head != nullptr) {
        cout << i++ << ". Nama: " << head->data.nama
        << ", NIM: " << head->data.nim
        << ", IPK: " << head->data.ipk << endl;
        head = head->next;
    }
    cout << endl;
}
```

Kode Program Lengkap

Berikut adalah implementasi struktur data Double Linked LIst menggunakan bahasa C++. Yang mencakup berbagai operasi dasar seperti penambahan node (add first, add specific, add last), penghapusan node (delete first, delete specific, delete last), serta fungsi untuk menampilkan seluruh isi Double Linked List:

program lengkap